Enhancing Search Privacy on Tor: Advanced Deep Keyword Fingerprinting Attacks and BurstGuard Defense

Chaiwon Hwang*
Ewha Womans University
Seoul, Republic of Korea
ifetayo@ewhain.net

Haeseung Jeon*
Ewha Womans University
Seoul, Republic of Korea
haeseungjeon@ewha.ac.kr

Jiwoo Hong
Ewha Womans University
Seoul, Republic of Korea
hjiwoo0914@ewhain.net

Hosung Kang
Ewha Womans University
Seoul, Republic of Korea
kanghsung717@ewhain.net

Nate Mathews
Rochester Institute of Technology
Rochester, USA
njm3308@rit.edu

Goun Kim
Ewha Womans University
Seoul, Republic of Korea
gowoon1230@ewhain.net

Se Eun Oh†
Ewha Womans University
Seoul, Republic of Korea
seoh@ewha.ac.kr

Abstract

This paper addresses the privacy risks associated with Keyword Fingerprinting (KF) on Tor, a process that involves inferring user-typed keywords through traffic analysis. Existing research has been inadequate in effectively fingerprinting keywords. To bridge this gap, we introduce Deep Keyword Fingerprinting (DKF), a novel approach that utilizes a large dataset, deep learning models, and adapted website fingerprinting features to significantly enhance the accuracy of predicting user-searched keywords. Additionally, we propose BurstGuard, an effective defense designed to disrupt keyword trace patterns and substantially defeat DKF, thereby improving upon state-of-the-art defense methods. Collectively, our study not only advances KF detection techniques but also strengthens defenses, significantly enhancing user privacy on Tor.

CCS Concepts

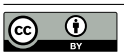
• **Security and privacy** → **Network security**; • **Computing methodologies** → **Machine learning**; **Artificial intelligence**; • **Networks** → **Network security**.

Keywords

Network Traffic Analysis, Keyword Fingerprinting, Privacy

ACM Reference Format:
Chaiwon Hwang, Haeseung Jeon, Jiwoo Hong, Hosung Kang, Nate Mathews, Goun Kim, and Se Eun Oh. 2025. Enhancing Search Privacy on Tor: Advanced Deep Keyword Fingerprinting Attacks and BurstGuard Defense. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '25), August 25–29, 2025, Hanoi, Vietnam.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3708821.3733914

*Equally credited authors.
†Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASIA CCS '25, Hanoi, Vietnam
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1410-8/25/08
https://doi.org/10.1145/3708821.3733914

1 Introduction

Search queries often contain significant amounts of private information, revealing sensitive details about an individual's interests, habits, health, and more. A notable example of potential privacy threats via search engines is the AOL search query leakage scandal [1]. To enhance user privacy and ensure anonymity, using an anonymous network like Tor has proven to be an effective strategy. Tor conceals the identities of both communicators, thereby preventing the destination server from determining the origin of the connection and effectively hiding users' identities.

Tor relays all communications through three proxy servers – named the entry guard, middle node, and exit node – which are randomly selected from thousands of volunteer routers globally. This architecture effectively conceals the connection between the client and the destination, as no single node involved in the communication has access to both pieces of information, forming what is known as a Tor circuit. Despite this, extensive research has demonstrated potential attacks on Tor that could compromise its anonymity by identifying the source or destination of the communications.

One such attack is Website Fingerprinting (WF), which identifies the websites visited by users based on the analysis of Tor traffic metadata, such as packet volume and timing information. These metrics reveal distinct traffic patterns depending on the destination servers. Recent literature on WF [17, 23, 24, 27] has demonstrated that it can infer the websites visited by users with over 95% accuracy. The high accuracy of these attacks has been driven by enhancements in fingerprinting models through the adoption of Deep Learning (DL) models, which have shown superior capabilities in feature extraction and classification compared to traditional Machine Learning (ML)-based WF models [9, 19, 30].

Moreover, Oh et al. [16] demonstrated that WF attacks can be extended to infer the keywords users type into search engines like

Google, Bing, and DuckDuckGo. This work establishes the feasibility of Keyword Fingerprinting (KF), which deduces keywords by analyzing Tor search query traffic – referred to as "keyword traces" – using Support Vector Machine (SVM) classifiers and well-crafted, task-specific feature sets.

Despite the severe user privacy threats posed by KF, research in this area has been relatively limited compared to WF. Thus, this paper revisits the KF problem using an up-to-date KF dataset and investigates the applicability of DL models to KF. Additionally, we devise a new defense technique to counteract DL-based KF attacks.

Through our extensive efforts in collecting a substantial KF training dataset, identifying new features better suited to KF, and adapting state-of-the-art WF models for KF, we propose a Deep Keyword Fingerprinting (DKF) model in this paper. To the best of our knowledge, DKF is the first KF attack model based on DL, utilizing the largest KF training dataset to date.

As we observed, search query traffic exhibits different patterns compared to website traffic, typically characterized by a higher overall volume of packets and a much larger proportion of incoming (from server to client) packets than outgoing (from client to server) packets. Consequently, the direct application of state-of-the-art WF models to KF could be limited in its fingerprinting capability. This paper demonstrates that DKF significantly enhances the capability of KF, surpassing even the performance of state-of-the-art DL-based WF models when applied to KF.

We also noted that all state-of-the-art WF attacks demonstrate poor performance in KF scenarios, achieving only 57% accuracy at best – much lower than their performance in WF. This indicates that KF scenarios are more challenging, as differentiating between keyword traces from a single website is more difficult than distinguishing between traces from different websites.

We further propose a novel KF defense named BurstGuard, which disrupts the pattern of incoming bursts, defined as more than two packets arriving consecutively in one direction. This strategy leverages the observation that incoming traffic dominates search query traces, a consequence of the bulky web content typically found on search result pages. Specifically tailored for keyword result traces, BurstGuard demonstrates superior defensive performance, significantly degrading DKF compared to existing methods.

The contributions of this paper are summarized as follows:

- **Up-to-date KF Dataset:** We introduce a large, current dataset for KF research, utilizing two search engines – Bing and DuckDuckGo – within the Tor network. Furthermore, to assess the impact of concept drift stemming from the evolving nature of webpage traces, we compiled an additional dataset with a time gap exceeding one year. This dataset is the most extensive and recent available for KF research.
- **More Effective KF Model with New Features:** Through a comprehensive feature analysis, we identified more effective features for KF by revisiting state-of-the-art WF and KF features [9, 16, 19, 23, 25, 27, 34]. We propose a new feature set by extending the Traffic Aggregation Matrix (TAM)[25], which effectively captures keyword traffic patterns by summarizing traffic information per short-time interval in noisy and bulky keyword traces. By fine-tuning the Deep Fingerprinting (DF) model[27], our DKF achieves an accuracy of 69%, significantly outperforming competing models. It shows a 41% higher accuracy than Tik-Tok [23], 48% higher than DF, and 55% higher than k-FP [9] on DuckDuckGo.
- **Analysis of KF Against Concept Drift:** We conducted the first comprehensive analysis of how shifts in trace distribution affect KF performance. This study examined the fingerprintability of keywords in the context of concept drift and explored various factors that contribute to differing levels of fingerprintability for each keyword.
- **DKF-Specific Defenses:** We introduce BurstGuard, the first KF defense specifically designed to conceal keyword traffic patterns by sending dummy incoming bursts whenever outgoing bursts occur. This innovative approach enhances privacy by disrupting traffic patterns in each short-time interval, thereby degrading DKF performance. When compared to well-known padding-based WF defenses such as FRONT [7] and WTF-PAD [13], BurstGuard demonstrates superior effectiveness, outperforming them by 23% and 11%, respectively, in defeating DKF.

2 Background and Related Works

In this section, we review previous studies on WF and KF attacks, as well as defenses against WF.

2.1 WF Attacks

Researchers have improved WF models by exploring various ML models and proposing new features. The SVM-based classifier is one of the most widely used in fingerprinting attacks [16, 19, 20, 31]. Panchenko et al. [20] achieved a 55% attack accuracy in WF by training an SVM model with packet volume, timing, and direction information. However, this approach incurred high computational costs, making it challenging to apply in real-world scenarios.

Wang et al. [30] proposed a WF attack using the k-Nearest Neighbors (k-NN) classifier, utilizing features including packet ordering, the number of incoming and outgoing Tor cells, and bursts. This approach achieved 91% accuracy in correctly classifying 100 pages in a closed-world scenario.

Panchenko et al. [19] introduced the CUMUL feature set for the SVM classifier, which computes the cumulative sum of packet sizes and includes categorical features such as the number of incoming and outgoing packets. This feature set is known for its computational efficiency and high performance, as the cumulative packet size sum sequence is inherently more consistent than the individual packet size sequence, with discrepancies decreasing as values approach the total packet size sum.

The k-fingerprinting (k-FP) model was introduced by Hayes et al.[9], utilizing a random forest classifier. With 150 hand-crafted features, it achieved high accuracy against top Alexa websites and Tor hidden services and modest accuracy against defended Tor traffic.

More recently, researchers [17, 24, 27] have investigated the applicability of DL. In particular, Rimmer et al. demonstrated the superior performance of a modified Stacked De-noising Auto-Encoder (SDAE) model, using the largest WF data collection available. Sirinam et al.[27] proposed the DF architecture, utilizing one-dimensional

Convolutional Neural Network (CNN) models with packet directional feature vectors. This architecture is recognized as one of the highest-performing WF models and has been widely adopted in various Tor fingerprinting studies [18, 23, 28].

Subsequently, Rahman et al.[23] introduced a novel directional timing feature, namely Tik-Tok, which underscored the importance of timing features in WF by enhancing the performance of the DF model. Bhat et al.[2] demonstrated the effectiveness of ensembling models, each fed with packet directional feature vectors and packet timestamp sequences.

Furthermore, Shen et al.[25] introduced a new type of WF feature, the Traffic Aggregation Matrix (TAM), which aggregates packet statistical information computed from each short interval, and fed this data into a CNN-based classifier. Their work shows robustness in fingerprinting traces defended by state-of-the-art WF defenses.

As this paper is the first to investigate the applicability of DL to KF problems, it is prudent to explore DL-based WF models in KF scenarios to inform the design of the DKF model. Additionally, we compare the performance of *k*-FP, CUMUL, DF, and Tik-Tok to DKF in Section 6.

2.2 KF Attacks

Given the success of WF on Tor, researchers are now investigating traffic analysis on Tor to infer user-typed keywords.

Oh et al. [16] proposed the first KF model, namely svmResp, by identifying new task-specific features focused exclusively on the largest incoming burst traffic, typically caused by downloading heavy web-embedded content. By adopting an SVM-based classifier that utilizes these burst-centric features, their approach demonstrates high KF accuracy.

Later, Yan et al. [34] proposed Wfin, an ensemble method based on Extra-Trees, which proved to be more efficient than SVM or Random Forest models. They utilized hand-crafted features and further refined their selection using Gini importance scores. Yan et al. [34] also confirmed Wfin's effectiveness in KF attacks across different client platforms, search engines, and feature sets.

In this paper, we compare the svmResp and Wfin models to DKF, and demonstrate their performance against an up-to-date KF dataset in Section 6.

2.3 WF Defenses

Given the high attack accuracy of WF models, researchers [3, 5–7, 10, 13, 32, 33] have devised several defenses to thwart these models. The first defense, BuFLO, introduced by Dyer et al. [6], disguises traffic by transmitting fixed-size packets at regular intervals over a specified duration. To address the high bandwidth overhead of BuFLO, Congestion-Sensitive BuFLO (CS-BuFLO)[3] was developed, followed by Tamaraw[32], which avoids unnecessary padding and dummy traffic to reduce bandwidth overhead.

Despite these improvements, these defenses continued to produce over 100% bandwidth and latency overhead. Furthermore, the latency overheads incurred by the defenses are particularly problematic as the end-user is directly impacted by increased page load times. To address this, Juarez et al. [13] proposed WTF-PAD, a defense that uses adaptive padding to insert only dummy packets to obfuscate features without modifying genuine traffic. This padding strategy uses a state machine that alternates between a BURST and GAP mode to adaptively insert fake bursts of traffic inside the gaps between real bursts within the sequence.

Following this, Gong et al.[7] proposed FRONT, another padding-only defense that adds dummy packets predominantly at the front of the traces without delaying packets. This strategy schedules dummy packets according to a Rayleigh distribution with a randomly generated parameterization per website visit. In addition to their low overheads, WTF-PAD and FRONT have the additional benefit of having a clear path towards implementation [21] in Tor's existing traffic padding solution. *Given these considerations, our defense, BurstGuard, continues to build upon this line of research, developing a new padding-only strategy that targets the unique burst characteristics of search engine queries.*

There is, however, another novel defense strategy to counter Tor traffic fingerprinting with little to no overheads. This strategy, which we call traffic splitting, involves sending packets over multiple paths. HyWF [10] splits traffic through two distinct network connections, such as via different internet service providers, while TrafficSliver [5] sends packets through multiple entry guards on Tor. This limits an adversary's visibility to a fraction of the traffic, significantly degrading WF performance without requiring padding or incurring bandwidth overhead. However, implementing these defenses requires significant changes to accommodate this new traffic routing design.

In this paper, we evaluate the performance of the DKF model against WTF-PAD, FRONT, and TrafficSliver defenses in Section 7.1 and 7.3.

3 Threat Model

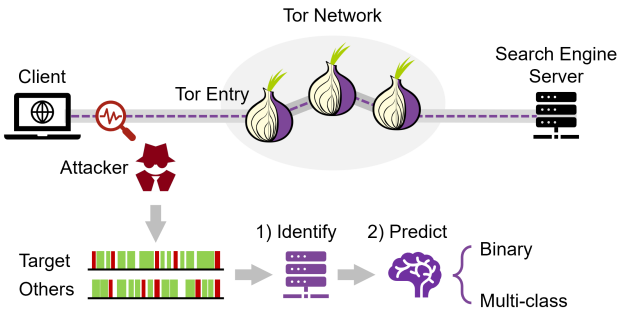


Figure 1: Threat model for Keyword Fingerprinting attacks.

As illustrated in Figure 1, we assume the presence of a passive, network-level adversary who monitors both inbound and outbound network traffic between the client and the Tor entry node (the first proxy) to identify search engine keywords typed by users.

This attack is executed in two steps: 1) identifying targeted search engine query traces, and 2) conducting either binary classification to differentiate monitored from unmonitored keywords, or multi-class classification to link each query trace to the specific keyword typed by the user. The first stage of the attack can be conducted with perfect accuracy, as demonstrated in previous literature [16].

This paper contributes to the second stage by enhancing state-of-the-art performance [16, 34], an approach we term the "Keyword Fingerprinting" attack.

The adversary maintains a database of search query traffic, which includes labels for search keywords and corresponding "keyword traces." Based on this search query traffic, the adversary can extract traffic metadata such as timestamps and sizes of encrypted packets. Utilizing a DL classifier pre-trained with this dataset, the attacker performs multi-class or binary classifications to identify monitored keyword traces and link them to specific keywords.

4 Data Collection

Previous research on KF has been hindered by the lack of comprehensive datasets that capture the real-world challenges, such as the gradual changes in website traffic patterns over time. As prior works in WF have noted [11], websites continuously evolve, leading to changes in their traces – a phenomenon known as "concept drift" [12]. This effect is pronounced in keyword traces due to frequent updates to search result pages. Consequently, periodic data collection is essential. To bridge this gap, we compiled the largest and most up-to-date keyword trace dataset. Our dataset not only encompasses a wide range of keywords from multiple search engines but also includes traces collected over an extended period, allowing us to analyze and address the impacts of concept drift on KF accuracy. This section outlines our data collection methodology and describes the datasets used in our study.

4.1 Data Collection Methodology

To automate data collection, we adapted an existing Tor Browser crawler [14], originally designed for WF dataset collection, to gather keyword traces. This crawler, employing `Selenium` to automate website visits and `dumpcap` for capturing traces on the Tor network, was extended to automate the typing of keywords by inputting each character sequentially into the search box. To simulate human typing patterns, we introduced random short-time delays between each character input. This delay is crucial as typing each character generates unique traffic patterns due to the dynamic autocomplete features provided by search engines to enhance user convenience. Consequently, we captured traffic that included both autocomplete suggestions and final search results, thereby ensuring richer data for KF attacks. Our modifications followed the guidelines provided by Oh et al. [16], and we further developed the data collection methods.

After each crawling session, we captured the network traffic and HTML files for further verification of the collected traces. Additionally, we took screenshots of the webpages for fingerprintability analysis, as detailed in Section 6.3. Given the substantial size of the trace files, which significantly prolonged server upload times, we preprocessed the data into text files, formatted according to the specifications outlined by Wang and Goldberg [31].

To verify the integrity of the collected traces, we examined the sizes of the HTML files. If their sizes were less than 265KB – a threshold we established based on manual inspection of randomly sampled pairs of traces and HTML files – traces were deemed faulty. In such cases, we reconstructed the Tor circuit and recollected the traces. Additionally, we filtered out collections whose packet counts were less than 100 to ensure a sufficient level of information on keyword traces.

Using `pyshark`, we extracted sequences of TCP packets, which were subsequently used to derive feature vectors, as detailed in Section 5.2 and 5.3. We employed four host machines, each operating eight virtual machines, and implemented a round-robin approach to distribute the collection of keyword traces across these machines. We captured the keyword traces for a period of 60 seconds, which was sufficient to accurately capture traffic involving dynamic web content, such as advertisements, embedded within web pages.

Using this methodology, we collected keyword traces from DuckDuckGo and Bing without requiring user sign-in. DuckDuckGo is the default search engine in the Tor Browser, and Bing has the second-highest global market share [4]. Unfortunately, we had to exclude Google from our study due to our inability to circumvent its detection mechanisms during our automated data collection, which we discuss further in Section 8.

We conducted our initial data collection from May to August 2023 using the university campus network. To explore the impact of concept drift, we subsequently gathered an additional dataset from December 2024 to January 2025, providing a time gap of over one year between the two data collection periods.

4.2 Dataset Overview

In this section, we describe the details of the dataset used in this paper.

Harvesting Keywords. We utilized the Keyword Tool website [29] to extract the top 273 frequently searched keywords as monitored keywords. Additionally, we gathered 110,000 unmonitored keywords from a dataset built by Oh et al. [16].

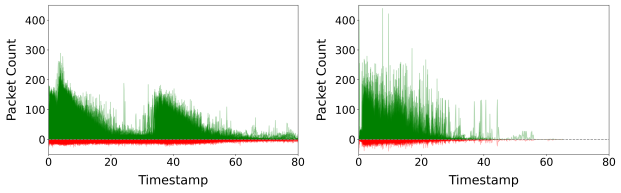
Bing Dataset ($\mathbf{Bing_{base}}$). For the monitored dataset, each of the 273 monitored keywords was queried 1,200 times across 50 batches in a round-robin manner. After removing faulty traces, we achieved 1,000 trace instances for 258 monitored keywords. For the unmonitored dataset, we collected one instance per keyword and, after filtering out abnormal traces, achieved a total of 93,443 unmonitored keyword traces.

DuckDuckGo Dataset ($\mathbf{DDG_{base}}$). We collected only the monitored dataset for DuckDuckGo. Similarly to Bing, each of the 273 monitored keywords was queried 1,400 times, distributed across 50 batches in a round-robin manner. As some classes lost samples due to filtering, we reorganized the dataset, limiting it to 1,000 instances per class to ensure balance.

Drift Dataset ($\mathbf{DDG_{drift}}$). We employed the same 50 batches in a round-robin manner and utilized identical keywords as those used in the collection of the DDG_{base} dataset to assess DKF against concept drift, resulting from a time gap of more than one year. After filtering out faulty traces, such as duplicates and incomplete traces, the final dataset comprised 273 classes, each containing 225 instances.

4.3 Dataset Analysis

In this section, we conducted an in-depth analysis of the AWF dataset [24], as well as the DDG_{base} and DDG_{drift} datasets, to examine the differences between KF traces and WF traces. Understanding



(a) DDG_{base} (b) AWF

Figure 2: Comparison of KF (DDG_{base}) and WF (AWF) traces: Visualization of a packet sequence histogram with incoming packet counts represented in green and outgoing packet counts in red.

these distinctions is crucial for designing effective feature engineering and fingerprinting models. Additionally, we utilized the dataset descriptions provided by Oh et al. [16] for further comparative analysis to understand the evolution of KF traces over time.

Table 1: Comparison of incoming bursts in WF (AWF) and KF (DDG_{base}, DDG_{drift}) datasets, based on all samples in the datasets.

Metric (Avg.)	AWF (WF)	DDG_{base} (KF)	DDG_{drift} (KF)
Trace Length (packet count)	**2759** (±2699)	**5232** (±1486)	**5796** (±1809)
Burst Length (packet count)	**5** (±1)	**10** (±3)	**8** (±4)
Largest Burst Length (packet count)	**36** (±17)	**61** (±16)	**65** (±22)
Burst Count	**447** (±367)	**542** (±337)	**692** (±407)
Burst Time Gap (seconds)	**0.08** (±0.80)	**0.05** (±0.10)	**0.06** (±0.09)
Largest Burst/Trace (%)	**3%**	**1%**	**1%**

As illustrated in Table 1, the KF datasets exhibit significantly longer trace and burst lengths — with bursts defined as more than two consecutive packets transmitted in one direction. Specifically, the DDG_{base} and DDG_{drift} datasets contain traces that are more than or nearly twice as long as those in the AWF dataset. Moreover, the shorter time intervals between consecutive bursts (Burst Time Gap) and a higher count of bursts indicate a prevalence of more frequent bursting in KF traces.

This pattern manifests as several peaks in KF traces, in contrast to WF traces, which typically show only a single peak. This observation is visualized in Figure 2. The observed pattern aligns with our experience that search result pages often contain more diverse content, resulting in more frequent communication between server and client compared to visits to the search engine's front page, which typically features more limited content such as a search box and links.

Furthermore, the DDG_{base} and DDG_{drift} datasets exhibit a relatively shorter length of the largest incoming burst, which contradicts the observations made by Oh et al. [16]. They reported that their collection of traces included a bulky incoming burst accounting for more than 50% of the entire trace, a finding that was pivotal in their feature design. However, such a pattern was not evident in our analysis of the DDG datasets, where the largest incoming burst constituted only 1% of the total packets, as detailed in Table 1. This discrepancy suggests a need for a novel feature design in KF work that does not rely on the size of the largest incoming burst. We will explore this further in Section 5.2.

5 DKF Settings

5.1 DKF Model

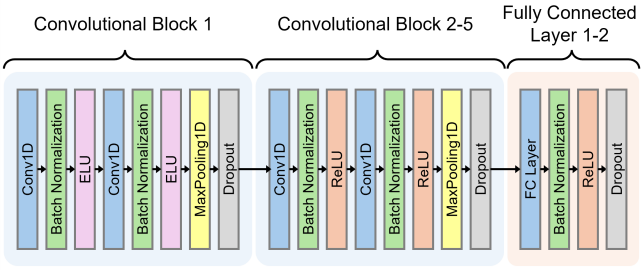


Figure 3: DKF Architecture.

The DF model [27] is recognized as one of the most effective WF models. Therefore, we initiated our research with this model and adapted it to create a DKF model. In KF, the inter-class variance is smaller because keyword traces from the same website server present less distinction compared to traces from different website servers. Consequently, our extension resulted in a slightly deeper architecture than that of DF, comprising five convolutional blocks. Each block contains two one-dimensional convolutional layers followed by a max pooling layer. This setup is further complemented by two fully connected layers, as depicted in Figure 3. The hyperparameters were also empirically selected based on the characteristics of keyword traces. Details on hyperparameter tuning will be provided in Section 5.4.

5.2 Revisiting Existing WF and KF Features

To enhance the effectiveness of KF attacks on the Tor network, we initially explored both WF and KF features.

We assume an example keyword trace, T_i, which consists of packet direction, d_i, packet arrival timestamps, t_i, and packet sizes, s_i, where $i \in \mathbb{N}$, $0 \leq i < n$, and the dataset contains n keyword traces in total. For T_i, we generate the packet direction sequence as $\{d_0, d_1, \ldots, d_{n-1}\}$, the packet timestamp sequence as $\{t_0, t_1, \ldots, t_{n-1}\}$, and the packet size sequence as $\{s_0, s_1, \ldots, s_{n-1}\}$. Lastly, the interval between consecutive packets has been proven to deliver useful signals for fingerprinting websites. The inter-packet delay (IPD) sequence is computed as $\{(t_1 - t_0), (t_2 - t_1), \ldots, (t_n - t_{n-1})\}$.

These features are well-documented within the WF literature [9, 19, 20, 24, 27, 30, 31]. Sirinam et al. [27] demonstrated through experimental results that packet size does not significantly enhance the performance of DL-based WF models, whereas directional feature vectors yielded better results. Similarly, Wang et al. [30] also

emphasized the utility of packet directional information in their feature vectors. This underscores the value of packet direction information as a crucial fingerprint in identifying websites through traffic analysis.

Concurrently, Oh et al. [16] developed a new feature set specifically tailored to the KF scenario, centered around the largest incoming burst. They engineered this feature set by calculating the total packet count and extracting the cumulative packet size sequence from the largest incoming bursts. This approach aligns with their observations that most keyword traffic includes a substantial volume of incoming burst traffic, which results from consecutive packets downloading web contents embedded in search result webpages.

More recently, advanced WF features have been developed as follows:

Tik-Tok. Rahman et al. [23] introduced a directional-timing feature called Tik-Tok, which combines packet direction with packet arrival timestamps. This combination is achieved by multiplying the timestamp of when a packet arrives by the packet's direction (outgoing indicated by +, incoming indicated by -). This feature vector, denoted as $\{d_0 \cdot t_0, d_1 \cdot t_1, \ldots, d_n \cdot t_n\}$, has demonstrated enhanced performance in DF models [23].

TAM. The TAM constructs a matrix by aggregating traffic information, such as the number of packets per subtrace, after segmenting the trace into multiple short-duration subtraces. This approach has proven robust, even against defended traffic, such as traces padded with dummy packets, because it effectively captures medium-grained traffic information within each short-time segment (window) of the trace. For instance, we assume an incoming subtrace, T_i^-, and an outgoing subtrace, T_i^+, by splitting T_i based on packet directions. Then, we divide T_i^- and T_i^+ into fixed-time intervals, w, and compute the number of packets in each w-second long window, denoted by $n(T_{i,j}^-)$ where j is the index of the window in T_i^-. In this manner, we generate two sequences of packet counts per window: W_i^+ and W_i^-, as detailed below.

$$W_i^+ = \{n(T_{i,j}^+) | j, m \in \mathbb{N}, 0 \leq j < m, m = \frac{d(T_i)}{w}\}$$

$$W_i^- = \{n(T_{i,j}^-) | j, m \in \mathbb{N}, 0 \leq j < m, m = \frac{d(T_i)}{w}\}$$

, where $d(T_i)$ is the total duration of T_i. Finally, they stacked W_i^+ and W_i^- into a single matrix.

However, none of these features resulted in superior performance for KF.

It is important to note that the state-of-the-art KF features proposed by Oh et al. [16] performed poorly due to concept drift, as keyword traces have evolved significantly over time. Oh et al. initially found that the largest incoming burst played a pivotal role in distinguishing one keyword trace from another, with their feature vectors extracted primarily based on this largest burst, which occupied 50% of the entire trace and effectively represented the characteristic of the trace. However, this characteristic is no longer present in the most recently collected keyword traffic data, underscoring the need for designing new features to address these changes.

Consequently, feature vectors focusing solely on the largest incoming bursts did not perform well with our dataset, as the traffic pattern has changed, including decreasing burst sizes over time. We concluded that feature engineering is necessary to account for potential shifts in packet distribution over time.

Furthermore, we also highlight that fine-grained packet-level features, such as packet directions, timestamps, sizes, and IPDs, were not helpful in identifying keyword traffic patterns. Instead, computing packet statistics such as the total packet count and packet size sum per short-time window could better capture the traffic patterns against highly variable and bulky keyword traces.

5.3 Winlap Feature

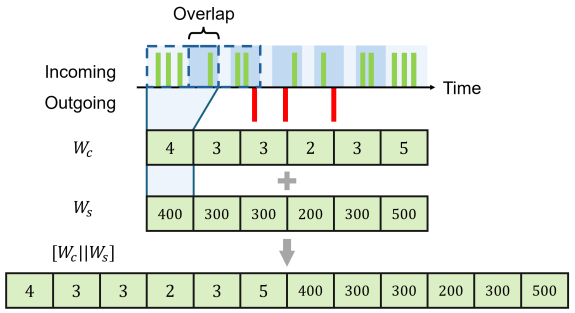


Figure 4: Winlap Feature.

Algorithm 1 Winlap Feature Generation Algorithm

Input: A trace, T_i, overlap ratio, o, window count, w_n, total duration, $d(T_i)$
Output: A feature vector, F_i
$W_c^i \leftarrow \{\}$ // Initialize empty count sum vector
$W_s^i \leftarrow \{\}$ // Initialize empty size sum vector
$w \leftarrow \frac{d(T_i)}{w_n}$ // Fixed-time intervals
$l \leftarrow \left(\frac{d(T_i)-w}{w\cdot(1-o)}\right) + 2$ // Total window count
$j \leftarrow 0$ // Window index in F_i
for p_i in T_i **do**
  Extract (t_i, d_i, s_i) from p_i
  $j \leftarrow \min(\left\lfloor \frac{t_i}{w\cdot(1-o)} \cdot l \right\rfloor, l-1)$

  // Only for incoming packets,
  if $d_i < 0$ **then**
    $W_{c,j}^i \leftarrow W_{c,j}^i + 1$ // Packet count sum
    $W_{s,j}^i \leftarrow W_{s,j}^i + s_i$ // Packet size sum

    // Only for overlapping packets,
    if $t_i < (w \cdot (1-o) \cdot j + w \cdot o)$ **then**
      $W_{c,j-1}^i \leftarrow W_{c,j-1}^i + 1$ // Packet count sum
      $W_{s,j-1}^i \leftarrow W_{s,j-1}^i + s_i$ // Packet size sum
    end if
  end if
end for
return $W_c^i || W_s^i$

As shown in Figure 2, KF traces exhibit more peaks in incoming traffic compared to WF traces. To effectively capture such characteristics, medium-grained features like the Traffic Aggregation Matrix (TAM) [25], which computes packet statistics per short-time interval, can be more effective than fine-grained features such as Tik-Tok.

Building on this insight, we have developed a new feature by enhancing TAM, tailoring it more specifically to the KF task.

First, given that keyword traces are less distinct between different classes (i.e., keywords) than website traces, we identified a need to amplify the TAM's capability to capture a more detailed traffic pattern in feature vectors. To achieve this, we divided the keyword traces into *overlapping* windows to more thoroughly capture the traffic pattern in each timeslot. It is also important to note that we consider only the incoming keyword trace, T_i^-, to extract the feature vector. After experimenting with various degrees of overlap, o, we divided a 60-second keyword trace into 2,000 subtraces by making each window overlap the next by 50%. This approach resulted in more effective feature vectors with a length of 4,000. Upon concatenation, the total feature length reached 8,000. We named this feature, "Winlap", which was then fed into the DKF models.

Furthermore, alongside computing the packet count for each window (W_c), we also calculated the sum of packet sizes per window (W_s) and concatenated W_c and W_s into a single feature vector. This feature generation process is illustrated in Figure 4. These adjustments led to a 6 percentage point increase in the accuracy of the DKF attack, enabling it to correctly identify 60 keywords. We detail the computation of Winlap feature vectors in Algorithm 1.

5.4 Hyperparameter Tuning

To mitigate the risk of overfitting and maximize the performance of the DKF model, it is essential to finely adjust the model parameters. We empirically selected the optimal combination of parameters that effectively differentiate keyword traces using a Bing dataset. The search spaces and chosen values for each hyperparameter are presented in Table 4.

Identifying the appropriate dropout rates within each convolutional block is crucial for the learning capability of the DKF model, as it reduces the likelihood of overfitting by randomly excluding nodes from the neural network during training. This approach helps the model generalize better to unseen data, thus enhancing DKF performance. We discovered that a dropout rate of 0.2 stabilizes the training process more effectively than rates of 0.1 or 0.3.

We also found that a lower batch size significantly enhances the learning capability of the DKF model. As a result, we selected a batch size of 32 for training the DKF model.

Additionally, we experimented with varying the number of convolutional blocks, as illustrated in Figure 3, and determined that five blocks were optimal for achieving the best performance. Increasing the number of blocks beyond five led to degraded performance and substantially longer running times, as evidenced by rising loss values and stagnating accuracies on both training and validation datasets.

Another key factor impacting performance is the selection of effective activation functions that scale gradients well, thereby enhancing learning capabilities. While the DF model [27] employs the ReLU function in each convolutional block except for the first, we tested ReLU, Leaky-ReLU (with an alpha value set at 0.01), and Swish activation functions in the convolutional layers from the second to the fifth blocks. Additionally, we confirmed that the use of ELU in the first convolutional block is beneficial. Our empirical analysis revealed no significant improvement in accuracy with the Leaky-ReLU function over the ReLU function, while the Swish function actually led to poorer DKF performance. Consequently, we adopted ReLU as the activation function for these convolutional blocks.

We also applied L2 regularization to the convolutional layers from the second to the fifth blocks to alleviate overfitting issues. However, this adjustment did not improve performance and even resulted in a slight drop in accuracy. Consequently, we decided to exclude regularization from these layers.

6 DKF Attack Evaluation

In this section, we evaluated the DKF model in both closed-world and open-world scenarios. In the closed-world scenario, we assume that users can only type keywords monitored by the adversary. Although this is an artificial setting, it provides a simplified experimental context to demonstrate the upper limit of DKF attack performance. In the open-world scenario, users may input *any* keywords, including those not monitored, leading to a simulation that is closer to real-world conditions. This scenario is crucial for understanding the practical performance of DKF attacks.

In these experiments, we also included comparisons to state-of-the-art KF models such as svmResp [16] and WFin [34] and WF models including Tik-Tok [23], DF [27], k-FP [9], and CUMUL [19].

6.1 Closed-World Experiments

For the comparisons to state-of-the-art WF and KF models, we evaluated these models using the $\text{Bing}_{\text{base}}$ and DDG_{base} datasets. To ensure fair comparisons, we tuned these models using our own dataset, $\text{Bing}_{\text{base}}$ and DDG_{base}. Table 3 displays the search spaces and chosen parameters for each model. We utilized the original implementations of svmResp, Tik-Tok, DF, k-FP, and CUMUL as found on GitHub [8, 15, 22, 26], and adapted the WFin model code [34] for KF on the Tor network, since the original work was focused solely on TLS-encrypted traces. This model prioritizes more significant features based on Gini importance scores, calculated using decision trees. Consequently, we re-selected the most important features to feed into the WFin model using our dataset. As CUMUL only provides the code for feature extraction, we additionally implemented the classification step by following the methodology described in the paper, using the `scikit-learn` library.

As shown in Table 2, the DKF model significantly outperformed all other models across various class sizes. Notably, DL-based models, such as DKF, Tik-Tok, and DF, secured the top three positions. This underscores that the effectiveness of KF benefits from the advanced feature extraction and classification capabilities of DL models, aligning with their impact on WF models [17, 23, 27]. Specifically, DKF achieved superior accuracy, surpassing Tik-Tok and DF by 14% and 25%, respectively, in correctly identifying 258 keywords in the $\text{Bing}_{\text{base}}$ dataset. Furthermore, when distinguishing 258 DuckDuckGo keyword traces, DKF improved accuracy by 41%

Table 2: DKF attack accuracy in the closed-world scenarios in $\text{Bing}_{\text{base}}$ and DDG_{base} dataset.

# of classes	DKF (%)		Tik-Tok (%)		DF (%)		k-FP (%)		Wfin (%)		svmResp (%)		CUMUL (%)	
	Bing	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*	*Bing*	*DDG*
60 classes	**85**	**82**	57	48	40	36	38	33	13	8	8	6	6	4
200 classes	**63**	**72**	47	32	37	24	31	16	7	2	4	2	3	1
258 classes	**59**	**69**	46	28	35	22	29	14	07	2	4	2	3	1

Table 3: Hyperparameter setting of state-of-the-art WF and KF models.

Model	Search Space	Chosen Parameter
DF(Tik-Tok)	Default Setting [23, 27].	epoch = 30 length = 5000 batch size = 128
k-FP	Default Settings [9].	n_estimators = 1000 cv = 5
Wfin	Default Settings [34].	n_estimators = 700 criterion = gini
CUMUL	$c : [2^{11} \ldots 2^{17}]$, $\gamma : [2^{-3} \ldots 2^{3}]$	$c = 2^{11}$, $\gamma = 2^{-3}$
svmResp	$c : [2^{-7} \ldots 2^{7}]$, $\gamma : [2^{-5} \ldots 2^{5}]$	Bing: $c = 2^{7}$, $\gamma = 2^{2}$ DDG: $c = 2^{7}$, $\gamma = 2^{-3}$

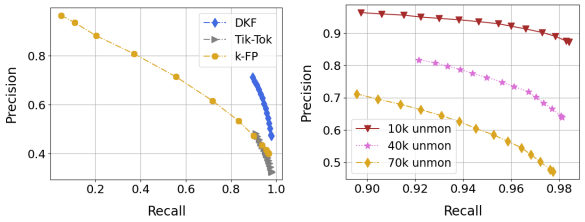
Table 4: Tuning details of hyperparameters in the DKF convolutional blocks.

Hyperparameters	Search Boundary	Chosen Parameter
Dropout Rate	[0.1, 0.2, 0.3]	0.2
Batch Size	[32, 64, 128]	32
Block Count	[4, 5, 6]	5
Activation Functions	ReLU, Leaky-ReLU, ELU, Swish	ELU, ReLU
Regularizations	L2, None	None

over Tik-Tok and 48% over DF. In contrast, models based on decision trees, such as k-FP and Wfin, exhibited relatively poor performance.

SVM-based models like svmResp and CUMUL formed the lowest-performing group. Particularly, svmResp – a state-of-the-art KF model evaluated on the Tor network – demonstrated significantly lower attack accuracy, which was the second lowest mainly due to its feature extraction approach. The features in svmResp are designed around the largest incoming burst, which Oh et al. [16] noted as comprising over 50% of packets. However, such large incoming bursts were absent in our dataset, with the largest burst accounting for only 3% of the total trace. This discrepancy led to a substantial decrease in the accuracy of svmResp when tested against our KF datasets. This outcome highlights the importance of periodic data collection and feature reengineering to adapt to concept drift in keyword traces.

To sum up, traditional ML- and DL-based WF models demonstrated poor or modest performance in KF tasks, significantly lower than their over 90% attack accuracies in WF [9, 19, 23, 27]. This study underscores the necessity of developing dedicated DL-based KF models for better fingerprinting accuracy and well-designed features to further enhance performance.



(a) Comparison of DKF, Tik-Tok, and k-FP. (b) P-R curve of DKF by varying unmonitored dataset.

Figure 5: Comparison of DKF performance with Tik-Tok and k-FP in the open-world scenarios.

6.2 Open-World Experiments

In this evaluation, we concentrated on three fingerprinting models – DKF, Tik-Tok, and k-FP – that have demonstrated high performance in closed-world scenarios. We constructed an open-world dataset consisting of 100 monitored keywords, each represented by 1,000 instances, and varied the size of the unmonitored keyword sets from 10,000 to 70,000. This setup enabled us to investigate how DKF's performance is influenced by the inclusion of unseen keywords. The open-world evaluation was structured around binary scenarios aimed at differentiating monitored keywords (labeled as 0) from unmonitored ones (labeled as 1). We exclusively used the $\text{Bing}_{\text{base}}$ dataset for this analysis.

We analyzed the DKF model's performance by plotting precision-recall (P-R) curves across different confidence thresholds, which were logarithmically scaled from 0 to 1. Since such thresholds were not applicable for k-FP, we implemented an alternative method for this model. In k-FP, the hamming distances to the k nearest neighbors were calculated to determine the final predicted labels at various thresholds. We defined a range between a minimum and a maximum distance value, scaled these values logarithmically, and subsequently plotted the P-R curves using randomly selected threshold values.

Figure 5(a) demonstrates the superior performance of the DKF model when compared to Tik-Tok and k-FP, highlighting its potential for practical application. Remarkably, DKF maintains robust performance even in scenarios characterized by a significant imbalance between monitored and unmonitored keywords, notably against 70,000 unmonitored keywords as opposed to merely 100

monitored classes. These findings, along with results from closed-world experiments, underscore the urgent need to develop defensive strategies capable of thwarting DKF-style attacks.

6.3 Concept Drift Experiments

As discussed previously, the performance of an adversary's fingerprinting model is expected to degrade as keyword search results or properties of the search engine changes over time. These dynamics necessitate periodic data collection and model retraining, a process that is resource-intensive. Given the high demand for computational resources, it is crucial to demonstrate the robustness of the DKF model over time, especially as trace distributions shift. To evaluate this robustness we train our fingerprinting models using the DDG_{base} and test on the DDG_{drift} dataset.

Table 5: DKF attack accuracy when evaluating with DDG_{base} and DDG_{drift}.

Model	Dataset Accuracy (%)	
	DDG_{base}	DDG_{drift}
DKF	**70**	**19**
Tik-Tok	28	6
DF	22	2
***k*-FP**	14	4

As illustrated in Table 5, trace distribution shift impacts all the classfiers significantly. However, DKF still outperforms other WF models.

Table 6: Top 10 best-performing keywords when evaluating with the DDG_{drift}.

№	Keyword	Accuracy (%)
1.	College Football Rankings	99
2.	Ohio State Football	93
3.	American Express	90
4.	Vrbo	90
5.	Google Classroom	86
6.	Espn	84
7.	Quest Diagnostics	83
8.	Southwest Airlines	81
9.	Xfinity Customer Service	80
10.	Taylor Swift	80

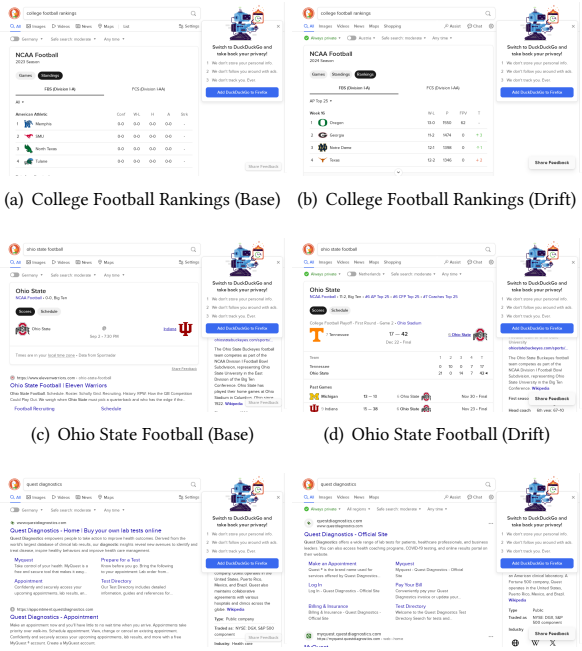
Furthermore, we have listed the top 10 keywords in Table 6 that are more robust against trace distribution shifts, showing greater preservation of keyword trace patterns over time. To investigate the characteristics of these keyword search results, which impact the stability of traffic patterns, we conducted further analyses by observing screenshots of the search result webpages. These observations are summarized as follows:

- **Static HTML Structure:** Keywords that yield search results with consistent HTML structures demonstrate clear stability over time. Examples include "College Football Rankings" and "Ohio State football," where the format of search results remains constant despite updates to content. For instance, differences between Figure 6(a) and 6(b), as well as between Figure 6(c) and 6(d), indicate changes in rankings and sports event results. However, the underlying HTML structure presenting this content has not evolved, contributing to more consistent search result traffic.
- **Static Search Result:** Some keywords inherently produce static search results due to their nature. For instance, terms like "American Express," "Google Classroom," and "VRBO" consistently yield site links from the same domains at the top of the search results list, which remain unchanged even after one year. Searching for "American Express" typically displays its official homepage, its Wikipedia introduction page, its Instagram profile, and similar links. These links rarely update over time as they are frequently accessed by many users.

We also performed a similar analysis on lower-accuracy keywords, and summarized their characteristics as follows:

- **Social Media Keywords:** Keywords such as "Reddit," "Facebook," "Pinterest," and "YouTube" yielded less than 1% accuracy. The search results for these terms frequently carry dynamic content, such as trending topics and news, which change in both content and HTML structure over time. A notable example is provided by Figure 7(a) and 7(b), which display the query results for "Reddit." While this keyword achieved 87.76% accuracy in the DDG_{base} dataset, it dropped to 0% accuracy in the DDG_{drift} dataset. This variability in search results leads to highly fluctuating traffic patterns over time.
- **Time-Sensitive Keywords:** Keywords such as "US election results," "US House elections 2022," "Veterans Day 2022," and "Labor Day 2022" yielded search results highly sensitive to temporal changes, leading to accuracies below 10%. For instance, a search for "US election results" delivered highlight news videos regarding the 2024 US elections (Figure 7(c)), an event that had not occurred prior to 2024, resulting in a completely different set of search results in Figure 7(d).
- **Location-Sensitive Keywords:** Query results for location-sensitive keywords like "Walgreens near me," "Jobs near me," and "Walmart near me," which achieved less than 9% accuracy, vary significantly based on the search location. For instance, the keyword associated with the search result shown in Figure 7(e) achieved 58.75% accuracy, while the same keyword, when reassessed one year later in Figure 7(f), yielded only 5.33% accuracy. This variability is due to the content of the query result webpages changing to reflect recommended places that vary greatly with the searcher's location.

These observations indicate that semantic dissimilarity and similarity in webpage contents significantly affect the consistency and inconsistency of keyword traces over time.

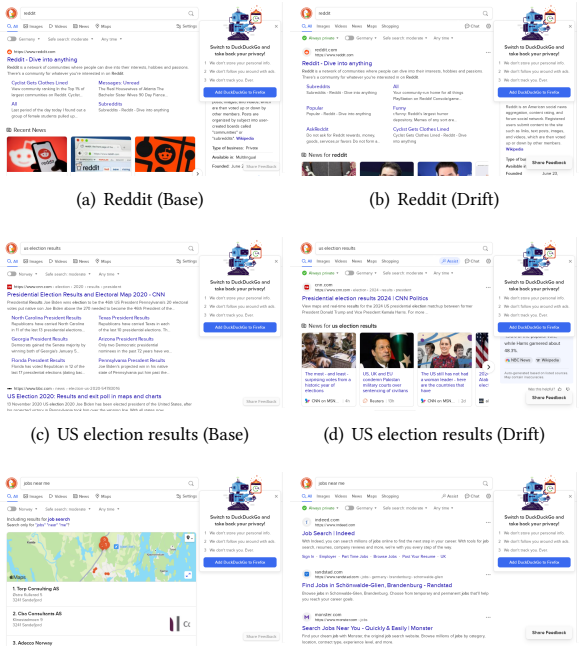


(a) College Football Rankings (Base) (b) College Football Rankings (Drift)

(c) Ohio State Football (Base) (d) Ohio State Football (Drift)

(e) Quest Diagnostics (Base) (f) Quest Diagnostics (Drift)

Figure 6: Webpage screenshots of search results for the top 10 better fingerprintable keywords against concept drift. Note that "Base" denotes keyword traces from DDG_{base} while "Drift" refers to keyword traces from DDG_{drift}.



(a) Reddit (Base) (b) Reddit (Drift)

(c) US election results (Base) (d) US election results (Drift)

(e) Jobs near me (Base) (f) Jobs near me (Drift)

Figure 7: Webpage screenshots of search results for less fingerprintable keywords against concept drift. Note that "Base" denotes keyword traces from DDG_{base} while "Drift" refers to keyword traces from DDG_{drift}.

7 DKF Evaluation Against Defenses

7.1 Revisiting Padding-Based WF Defenses

Experimental Settings. Although Oh et al. [16] demonstrated the effectiveness of several traditional WF defenses such as Tamaraw [32] and BuFLO [6], their evaluations were limited to older, heavyweight defenses that result in significant bandwidth overhead. In response, this study explores more recent WF defenses, including WTF-PAD [13], and FRONT [7]. These defenses offer lower or no bandwidth overhead while still maintaining an adequate level of defensive capability. This enhancement in efficiency makes them more viable for practical deployment. We specifically assess their effectiveness against DKF attacks to determine if these newer defenses can achieve both operational efficiency and effective security. For this evaluation, we utilized the DDG_{base} dataset, which includes 258 monitored classes, each with 1,000 instances.

For FRONT, we randomly inserted between a minimum of 3 and a maximum of 2,100 dummy packets in both incoming and outgoing directions, resulting in a 31% bandwidth overhead.

For WTF-PAD, traces were extracted using two methods: normal and normal_rcv. The primary distinction between these settings is that normal considers only the distribution of outgoing traffic, whereas normal_rcv accounts for both incoming and outgoing traffic distributions when identifying the locations for dummy bursts. In our settings, both approaches resulted in a bandwidth overhead of 38%.

To make a comparison, we will evaluate the performance of each defense under the same bandwidth overhead conditions. The bandwidth overhead for each defense is computed across n number of traces. When an undefended packet size sequence, $s_1, s_2, ..., s_m$, is input into a given defense algorithm, which subsequently processes and outputs a defended packet size sequence, $p_1, p_2, ..., p_l$, the bandwidth overhead is calculated for each defense using $\frac{1}{n}\sum_{i=1}^{n}\frac{\sum_j s_j}{\sum_k p_k}$. This equation measures the relative increase in data transmitted due to the defense mechanism, quantifying the efficiency and cost of each defense approach in terms of bandwidth consumption.

Results. Based on Table 7, despite its performance degradation under the WF defense, DKF still managed modest performance across FRONT and WTF-PAD defenses, significantly outperforming both Tik-Tok and k-FP in all scenarios. Notably, under FRONT and WTF-PAD, DKF achieved high accuracies of 62% and 49-50%, respectively.

In contrast, the performance of Tik-Tok and k-FP deteriorated considerably, suggesting that their reliance on packet-level features such as packet timing and hand-crafted features derived solely from the entire trace fails to effectively capture traffic patterns in padded keyword traces. Conversely, Winlap demonstrated its utility by maintaining informative traffic patterns within short-time intervals, which supported the modest performance of DKF.

Overall, DKF demonstrated robust capabilities in effectively classifying keywords, even against traffic defended by padding with

Table 7: The KF evaluation on defended keyword traffic considers varying settings of bandwidth overheads. The lowest accuracy at the same bandwidth overhead is highlighted in bold.

Bandwidth Overhead (%)	0	31		38		
Defense Method	None	BurstGuard	FRONT	BurstGuard	WTF-PAD	WTF-PAD$_{rcv}$
DKF Accuracy (%)	69	**39**	62	**38**	49	50
Tik-Tok Accuracy (%)	28	6	2	3	8	8
k-FP Accuracy (%)	14	10	8	9	9	9

dummy packets. It significantly outperformed Tik-Tok and k-FP, with margins wider than those reported in Section 6.

7.2 Introducing a New Defense

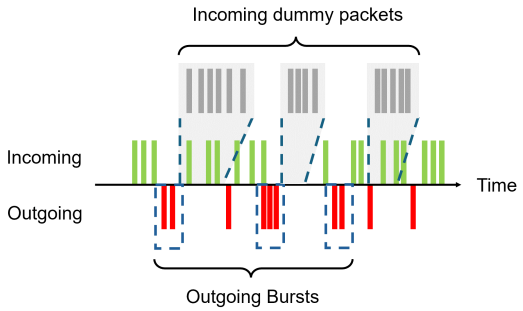


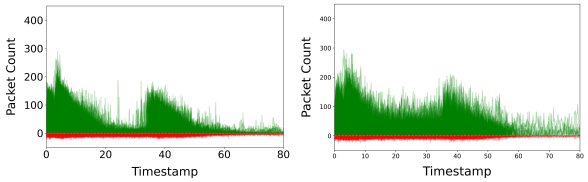
Figure 8: Overview of BurstGuard defenses.

Design. The existing padding-based defense methods, such as FRONT and WTF-PAD, failed to effectively degrade the performance of the DKF model. Therefore, in this study, we aim to design more effective padding schemes that obscure the keyword trace pattern, potentially threatening DKF's performance.

As discussed in Section 4.3, our new defense strategy is based on the observation that keyword traces now exhibit more frequent and larger bursts, particularly with the largest burst constituting about 1% of the entire trace. This increase in burst frequency can likely be attributed to enhanced bi-directional communication between the client and server, driven by the uploading of various web-embedded contents on search result pages.

Our new defense proposal is outlined in Algorithm 2 and visualized in Figure 8.

To mask the frequent incoming burst patterns and the associated short-time window patterns, we have developed a strategy that schedules incoming dummy bursts, b', whenever outgoing bursts are detected. The number of dummy packets in each burst, denoted by n, is randomly selected from a range between 1 and N, effectively obfuscating the burst patterns from potential attackers. The parameters n and N are hyperparameters that should be carefully adjusted to optimize the balance between minimizing bandwidth overhead and maximizing defense effectiveness. In our experiments, we set N to 21 and 26, which resulted in bandwidth overheads of 31% and 38%, respectively, allowing for comparative analysis with other defense mechanisms.



(a) DDG$_{\text{base}}$ (0%) (b) BurstGuard (38%)

Figure 9: Visualizations of "US election results" Keyword traces from (DDG$_{\text{base}}$), with no defense and with defense.

To properly schedule dummy bursts to mimic real KF traces, we constructed histograms using the IPDs, representing the time intervals between two consecutive packets, and packet sizes, extracted from the 10 sample traces of DDG$_{base}$. These histograms are denoted as H_{ipd} and H_{size}. It is advisable to compute these histograms using a substantial, publicly accessible KF dataset. By calculating the mean and standard deviation of continuous IPD values from KF traces, we fit a Gaussian distribution across 1,000 bins. Additionally, we constructed a histogram by directly counting each of the specified bins based on packet sizes extracted from KF traces.

Whenever an outgoing burst is detected, the delay, δ, is randomly chosen from the lowest 20% of H_{ipd}, and an incoming dummy packet, p', consisting of the timestamp, direction, and size – represented as (t, d, s) – is inserted at that timestamp. This procedure is repeated n times. As BurstGuard exclusively schedules incoming bursts, d is always set to -1, indicating an incoming direction, and s is randomly selected based on H_{size}.

We have named this defense "BurstGuard," as it is designed to obscure burst patterns in keyword traces that occur when downloading web-embedded objects on search result pages.

Results. As shown in Table 7, BurstGuard successfully degraded DKF performance and proved more effective than traditional WF defenses in thwarting DKF attacks. However, its performance was comparable to other defenses when facing Tik-Tok and k-FP. Since Tik-Tok analyzes packet timestamps and directions, and k-FP utilizes hand-crafted features such as total packet statistics and directional packet arrival timestamp feature vectors, merely scheduling incoming dummy packets in response to outgoing bursts may not effectively obscure such fine-grained features. Nonetheless, the pronounced effectiveness of BurstGuard against the DKF model is likely due to its specific focus on obfuscating incoming burst patterns. This strategy effectively disrupts several high peaks of incoming keyword traffic (Figure 2(a)), thus disrupting window-level keyword traffic patterns and establishing a more robust defense.

Algorithm 2 BurstGuard Defense Algorithm

Input: A trace, T_i
Output: A simulated trace, T_i'
$T_i' \leftarrow \{\}$ // Initial empty list
$H_{ipd} \leftarrow$ IPD histogram
$H_{size} \leftarrow$ Size histogram
N // Number of dummy packet
$t_p \leftarrow 0$ // Timestamp of previous packet
s_d // Size of dummy packet
for p_i **in** T_i **do**
  extract (t_i, d_i, s_i) from p_i
  $t_p \leftarrow t_i$
  $T_i' \leftarrow T_i' \cup p_i$
  pick n randomly from $[1, N+1)$
  if d_i **is** true **and** $p_i \in$ burst **then**
    $b_i' \leftarrow \{\}$ // Dummy burst
    for j **in** range(n) **do**
      $p_{i,j}' \leftarrow ()$ // Dummy packet
      pick δ from the bottom 20% of H_{ipd}
      pick s_d from the H_{size}
      $p_{i,j}' \leftarrow (t_i + \delta, -1, s_d)$
      $b_i' \leftarrow b_i' \cup p_{i,j}'$
    end for
    $T_i' \leftarrow T_i' \cup b_i'$
  end if
end for

Before and after applying the BurstGuard defense algorithm, which results in a bandwidth overhead of 38%, we visualized the trace of the keyword "US election results" in Figure 9. Dummy packets are strategically inserted between two peaks of incoming traffic and at the end of the trace. This placement makes each window pattern less distinct, thereby further degrading the performance of the DKF model.

However, DKF still achieved nearly 40% attack accuracy in a 38% overhead setting, indicating that more effective defenses are needed to thoroughly thwart DKF-style attacks. We leave the further enhancement of BurstGuard and the exploration of additional strategies as avenues for future work.

7.3 Exploring Traffic Splitting Defenses.

Although BurstGuard was the most effective padding-only defense in diminishing the performance of our DKF attack, our defense method still introduces some bandwidth overhead. In contrast, traffic-splitting defenses offer a padding-free alternative, if the implementation challenges can be overcome. Therefore, we benchmark DKF against TrafficSliver [5].

The TrafficSliver approach transmits packets across n paths created by n entry guards, allowing the attacker to observe only $\frac{1}{n}$ of the total transmitted packets, significantly reducing the number of packets monitored. Increasing n enhances security without adding bandwidth overhead, as no dummy packets are inserted, but adds complexity to deployment.

TrafficSliver supports various path selection algorithms, including round robin, random, weighted random (WR), and batched weighted random (BWR). We selected the BWR scheme for our experiments, as it was identified as the most effective in their proposal. Configurations with $n = 3$ and $n = 5$ paths were tested to balance security and utility. Under the BWR scheme, traffic is sent to a path in batches of 50 to 70 packets. We assigned equal probabilities when selecting a path.

Table 8: KF evaluation on defended keyword traffic by TrafficSliver.

Models	TrafficSliver	
n	**3**	**5**
DKF Accuracy (%)	**18**	**10**
Tik-Tok Accuracy (%)	3	2
k-FP Accuracy (%)	2	2

As shown in Table 8, DKF achieved higher accuracy compared to Tik-Tok and k-FP, yet its accuracy was significantly reduced to between 10–18%. This indicates that traffic splitting effectively obscures keyword traffic patterns, mirroring its demonstrated efficacy in standard WF scenarios. These findings suggest that traffic splitting defenses would also be effective at preventing KF if the strategy is implemented into the Tor network.

8 Discussion

Google Crawling Challenges. While attempting to crawl the Google search engine, we frequently encountered CAPTCHA challenges, with only about 80% of visits succeeding. This issue appeared to stem from Google applying increased monitoring to known Tor exit relay IP addresses. In an attempt to resolve this issue, we explored the following methods, without success:

(1) **Reconstructing Tor Circuits**: Following the approach of Oh et al. [16], we periodically rebuilt the Tor circuit to obtain a new exit node. While this occasionally bypassed CAPTCHA challenges, successes were too infrequent to reliably scale to collect a complete dataset.
(2) **Using Chrome Extensions**: We explored CAPTCHA avoidance Chrome extensions. However, since our data collection was performed on a headless server, these extensions were incompatible.
(3) **Google Account Sign-Ins**: We created multiple Google accounts and signed in to bypass CAPTCHA. While effective, this approach introduced bias in search results, limiting the diversity of query outcomes and keyword traces.

Therefore, given Google's stringent CAPTCHA enforcement, we recommend that future research efforts design their data collection systems with integrated CAPTCHA bypass mechanisms from the outset.

Sensitive Keywords. While this paper focused on frequently searched keywords, the privacy implications of KF are particularly severe when targeting highly sensitive queries, such as those related to personal health, political dissent, or stigmatized topics. These keywords often reveal intimate or private details, making them critical to study for both adversarial potential and defense development. However, it is unclear if the network traffic properties

of sensitive keywords follow that of frequently searched keywords. As our results with DKF demonstrated, some keywords are particularly easy to identify. Future work should investigate how the traffic patterns of sensitive keywords differ from frequently searched keywords, evaluate the robustness of KF models in these contexts, and consider tuning BurstGuard for these keywords.

Richer User Behavior Modeling. In this study, we simulated user typing behavior by adding a random time gap between key presses. However, real-world user behavior during web searching is more complex, including aspects such as varied typing speeds, pausing mid-query, deleting and retyping portions of text, or using autofill suggestions. Moreover, while collecting the dataset, we discovered the pop-ups for the cookie consent within certain regions. In this case, the user can take extra actions such as closing the pop-ups, which can change the patterns of the trace. Therefore, incorporating these behaviors into KF research by simulating more realistic traces could significantly improve the realism of evaluations. Future work could explore how these dynamic behaviors impact KF attack accuracy and robustness.

9 Conclusion

In this paper, we introduce Deep Keyword Fingerprinting (DKF), a state-of-the-art model that significantly advances the accuracy of keyword prediction on Tor by leveraging novel features and the largest KF dataset to date. DKF achieves up to 69% accuracy in multi-class classification tasks, outperforming prior WF and KF models by up to 48 percentage points. Moreover, DKF demonstrates greater robustness across closed-world and open-world scenarios, maintaining effective performance against large-scale unmonitored keyword datasets.

To counter this heightened threat, we propose BurstGuard, a novel and targeted defense mechanism specifically designed to disrupt keyword trace patterns. By injecting incoming dummy bursts in response to outgoing burst traffic, BurstGuard effectively reduces DKF's accuracy by up to 30 percentage points, significantly outperforming existing defenses such as WTF-PAD and FRONT. Our findings not only advance KF models and defense techniques but also enhance user anonymity on Tor, providing actionable guidelines for developing more effective defenses against DKF.

Availability

Our implementations of DFK, BurstGuard, and the KF dataset are available at https://github.com/AISec-EWHA/deep-keyword-fingerprinting, and the implementation of the Tor crawler is available at https://github.com/AISec-EWHA/KF-tbcrawler.

Acknowledgments

We would like to thank our anonymous reviewers for helpful comments. This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2022-00155966, Artificial Intelligence Convergence Innovation Human Resources Development (Ewha Womans University)), and by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2023-00222385).

References

[1] Michael Barbaro and Tom Zeller Jr. 2006. A Face Is Exposed for AOL Searcher No. 4417749. https://www.nytimes.com/2006/08/09/technology/09aol.html. Accessed: 30-Jul-2024.
[2] Sanjit Bhat, David Lu, Albert Kwon, and Srinivas Devadas. 2018. Var-CNN: A data-efficient website fingerprinting attack based on deep learning. *arXiv preprint arXiv:1802.10215* (2018).
[3] Xiang Cai, Rishab Nithyanand, and Rob Johnson. 2014. Cs-buflo: A congestion sensitive website fingerprinting defense. In *Proceedings of the 13th Workshop on Privacy in the Electronic Society*. 121–130.
[4] A. Chris. 2024. Top 10 search engines in the world. https://www.reliablesoft.net/top-10-search-engines-in-the-world/. Accessed: 30-Jul-2024.
[5] Wladimir De la Cadena, Asya Mitseva, Jens Hiller, Jan Pennekamp, Sebastian Reuter, Julian Filter, Thomas Engel, Klaus Wehrle, and Andriy Panchenko. 2020. Trafficsliver: Fighting website fingerprinting attacks with traffic splitting. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 1971–1985.
[6] Kevin P Dyer, Scott E Coull, Thomas Ristenpart, and Thomas Shrimpton. 2012. Peek-a-boo, i still see you: Why efficient traffic analysis countermeasures fail. In *2012 IEEE symposium on security and privacy*. IEEE, 332–346.
[7] Jiajun Gong and Tao Wang. 2020. Zero-delay lightweight defenses against website fingerprinting. In *29th USENIX Security Symposium (USENIX Security 20)*. 717–734.
[8] Jamie Hayes and George Danezis. [n. d.]. *k-fingerprinting: A Robust Scalable Website Fingerprinting Technique*. doi:jhayes14/k-FP
[9] Jamie Hayes and George Danezis. 2016. k-fingerprinting: A robust scalable website fingerprinting technique. In *25th USENIX Security Symposium (USENIX Security 16)*. 1187–1203.
[10] Sébastien Henri, Gines Garcia-Aviles, Pablo Serrano, Albert Banchs, and Patrick Thiran. 2020. Protecting against Website Fingerprinting with Multihoming. *Proceedings on Privacy Enhancing Technologies* 2020 (2020), 89 – 110.
[11] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Rachel Greenstadt. 2014. A critical evaluation of website fingerprinting attacks. In *Proceedings of the 2014 ACM SIGSAC conference on computer and communications security*. 263–274.
[12] Marc Juarez, Sadia Afroz, Gunes Acar, Claudia Diaz, and Rachel Greenstadt. 2014. A critical evaluation of website fingerprinting attacks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*. 263–274.
[13] Marc Juarez, Mohsen Imani, Mike Perry, Claudia Diaz, and Matthew Wright. 2016. Toward an efficient website fingerprinting defense. In *Computer Security–ESORICS 2016: 21st European Symposium on Research in Computer Security, Heraklion, Greece, September 26-30, 2016, Proceedings, Part I 21*. Springer, 27–46.
[14] Nate Mathews. [n. d.]. *Tor-Browser-Crawler*. doi:notem/tor-browser-crawler
[15] Se Eun Oh, Shuai Li, and Nicholas Hopper. [n. d.]. *Fingerprinting Keywords in Search Queries over Tor*. doi:KeywordFingerprinting/KF
[16] Se Eun Oh, Shuai Li, and Nicholas Hopper. 2017. Fingerprinting Keywords in Search Queries over Tor. *Proc. Priv. Enhancing Technol.* 2017, 4 (2017), 251–270.
[17] Se Eun Oh, S Sunkam, and N Hopper. 2019. p-fp: Extraction, classification, and predication of website fingerprints. In *Proc. of PETS*, Vol. 19.
[18] Se Eun Oh, Taiji Yang, Nate Mathews, James K Holland, Mohammad Saidur Rahman, Nicholas Hopper, and Matthew Wright. 2022. DeepCoFFEA: Improved flow correlation attacks on Tor via metric learning and amplification. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1915–1932.
[19] Andriy Panchenko, Fabian Lanze, Jan Pennekamp, Thomas Engel, Andreas Zinnen, Martin Henze, and Klaus Wehrle. 2016. Website Fingerprinting at Internet Scale.. In *NDSS*.
[20] Andriy Panchenko, Lukas Niessen, Andreas Zinnen, and Thomas Engel. 2011. Website fingerprinting in onion routing based anonymization networks. In *Proceedings of the 10th Annual ACM Workshop on Privacy in the Electronic Society* (Chicago, Illinois, USA) *(WPES '11)*. Association for Computing Machinery, New York, NY, USA, 103–114. doi:10.1145/2046556.2046570
[21] Tobias Pulls and Ethan Witwer. 2023. Maybenot: A Framework for Traffic Analysis Defenses. In *Proceedings of the 22nd Workshop on Privacy in the Electronic Society* (Copenhagen, Denmark) *(WPES '23)*. Association for Computing Machinery, New York, NY, USA, 75–89. doi:10.1145/3603216.3624953
[22] Mohammad Saidur Rahman, Payap Sirinam, Nate Mathews, Kantha Girish Gangadhara, and Matthew Wright. [n. d.]. *Tik-Tok: The utility of packet timing in website fingerprinting attacks*. doi:msrocean/Tik_Tok
[23] Mohammad Saidur Rahman, Payap Sirinam, Nate Mathews, Kantha Girish Gangadhara, and Matthew Wright. 2020. Tik-Tok: The utility of packet timing in website fingerprinting attacks. *Proceedings on Privacy Enhancing Technologies* 2020, 3 (2020), 5–24.
[24] Vera Rimmer, Davy Preuveneers, Marc Juarez, Tom Van Goethem, and Wouter Joosen. 2018. Automated Website Fingerprinting Through Deep Learning. In *Proceedings 2018 Network and Distributed System Security Symposium*. The Internet Society. doi:10.14722\/ndss.2018.23105 25th Annual Network and Distributed System Security Symposium, NDSS 2018 ; Conference date: 18-02-2018 Through 21-11-2018.

[25] Meng Shen, Kexin Ji, Zhenbo Gao, Qi Li, Liehuang Zhu, and Ke Xu. 2023. Subverting website fingerprinting defenses with robust traffic representation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 607–624.

[26] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. [n. d.]. . doi:deep-fingerprinting/df

[27] Payap Sirinam, Mohsen Imani, Marc Juarez, and Matthew Wright. 2018. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 1928–1943.

[28] Payap Sirinam, Nate Mathews, Mohammad Saidur Rahman, and Matthew Wright. 2019. Triplet fingerprinting: More practical and portable website fingerprinting with n-shot learning. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 1131–1148.

[29] Keyword Tool. 2025. Keyword Tool: Online Keyword Analysis Tool. Retrieved Jan 21, 2025 from https://keywordtool.io/

[30] Tao Wang, Xiang Cai, Rishab Nithyanand, Rob Johnson, and Ian Goldberg. 2014. Effective Attacks and Provable Defenses for Website Fingerprinting. In *23rd USENIX Security Symposium (USENIX Security 14)*. USENIX Association, San Diego, CA, 143–157.

[31] Tao Wang and Ian Goldberg. 2013. Improved website fingerprinting on Tor. In *Proceedings of the 12th ACM Workshop on Workshop on Privacy in the Electronic Society* (Berlin, Germany) *(WPES '13)*. Association for Computing Machinery, New York, NY, USA, 201–212. doi:10.1145/2517840.2517851

[32] Tao Wang and Ian Goldberg. 2014. *Comparing website fingerprinting attacks and defenses*. Technical Report. Technical Report 2013-30, CACR, 2013.

[33] Tao Wang and Ian Goldberg. 2017. Walkie-Talkie: An efficient defense against passive website fingerprinting attacks. In *26th USENIX Security Symposium (USENIX Security 17)*. 1375–1390.

[34] Junhua Yan, Hasan Faik Alan, and Jasleen Kaur. 2020. Fingerprinting Search Keywords over HTTPS at Scale. *arXiv preprint arXiv:2008.08161* (2020).